1. Setup and Execution Instructions

This section provides the steps required to reproduce the project locally, run the model training pipeline, view MLflow experiment tracking, serve the model through FastAPI, containerize the API, deploy it to Kubernetes, and run the monitoring stack.

1.1 Clone the Repository

```
git clone https://github.com/Ankita-BITS/heart-disease-mlops
```

1.2 Create and Activate a Virtual Environment

For Windows PowerShell:

```
py -3.11 -m venv venv
.\venv\Scripts\Activate.ps1
```

For Windows Command Prompt:

```
py -3.11 -m venv venv
venv\Scripts\activate
```

For macOS/Linux:

```
python3.11 -m venv venv
source venv/bin/activate
```

1.3 Install Project Dependencies

```
python -m pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
```

The project dependencies include packages for data processing, model training, experiment tracking, API serving, testing, linting, and monitoring.

1.4 Download and Prepare the Dataset

The dataset is downloaded using the `ucimlrepo` package and saved into the project data folders.

```
python src/download_data.py
```

Expected outputs:

```
data/raw/heart_disease_raw.csv
data/processed/heart_disease_clean.csv
```

1.5 Run Exploratory Data Analysis

The EDA notebook is located at:

```
notebooks/01_eda.ipynb
```

This notebook includes:

- Missing value analysis
- Class balance visualization
- Feature distributions
- Correlation heatmap
- Feature relationship analysis

EDA figures are saved under:

```
reports/figures/
```

1.6 Train and Compare Models

Run the training script:

```
python src/train.py
```

The training script performs preprocessing, model training, hyperparameter tuning, cross-validation, model comparison, MLflow logging, and model packaging.

The models evaluated include:

- Logistic Regression
- Random Forest Classifier

Expected outputs:

```
model/heart_disease_pipeline.joblib
model/feature_metadata.json
model/sample_input.json
reports/model_comparison.csv
reports/figures/
```

1.7 View MLflow Experiment Tracking

The project uses MLflow to track model experiments, hyperparameters, metrics, plots, and trained model artifacts.

Start the MLflow UI:

```
python -m mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --
port 5000
```

Then open:

```
http://127.0.0.1:5000
```

The MLflow experiment is named:

```
heart_disease_classification
```

In the MLflow UI, review:

- Logistic Regression run
- Random Forest run
- Accuracy, precision, recall, F1-score, and ROC-AUC metrics
- Confusion matrix artifacts
- ROC curve artifacts
- Saved model artifacts
- Final selected model run

Screenshots from MLflow are included in the report as experiment tracking evidence.

1.8 Run Local Prediction from the Saved Pipeline

After training, test the packaged model pipeline using the sample input file:

```
python src/predict.py --input model/sample_input.json
```

The script loads the saved preprocessing and model pipeline and returns the predicted class and confidence probability.

1.9 Run Unit Tests and Lint Checks

Run unit tests:

```
pytest tests -v
```

Run lint checks:

```
flake8 src tests
```

These checks are also executed automatically in the GitHub Actions CI workflow.

1.10 Run the FastAPI Application Locally

Start the API locally:

```
python -m uvicorn api.app:app --host 127.0.0.1 --port 8000 --reload
```

Open the Swagger documentation:

```
http://127.0.0.1:8000/docs
```

Health check endpoint:

```
http://127.0.0.1:8000/health
```

Prediction endpoint:

```
http://127.0.0.1:8000/predict
```

Metrics endpoint:

```
http://127.0.0.1:8000/metrics
```

Example prediction request body:

```
{
  "age": 63,
  "sex": 1,
  "cp": 3,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
  "thal": 1
}
```

1.11 Build and Run the Docker Container

Build the Docker image:

```
docker build -t heart-disease-api:latest .
```

Run the container:

```
docker run -d -p 8000:8000 --name heart-disease-api-container heart-disease-
api:latest
```

Check that the container is running:

```
docker ps
```

Open:

```
http://127.0.0.1:8000/docs
```

Stop and remove the container when finished:

```
docker stop heart-disease-api-container
docker rm heart-disease-api-container
```

1.12 Deploy to Local Kubernetes

The project includes Kubernetes manifests under:

```
deployment/
```

Apply the Kubernetes deployment and service:

```
kubectl apply -f .\deployment\namespace.yaml
kubectl apply -f .\deployment\deployment.yaml
kubectl apply -f .\deployment\service.yaml
```

Check the deployed resources:

```
kubectl get all -n heart-disease
```

Wait for the deployment to become available:

```
kubectl wait --for=condition=available deployment/heart-disease-api -n heart-
disease --timeout=120s
```

If direct service access is not available, use port forwarding:

```
kubectl port-forward -n heart-disease service/heart-disease-api-service 8000:8000
```

Then open:

```
http://127.0.0.1:8000/docs
```

Check Kubernetes logs:

```
kubectl logs -n heart-disease deployment/heart-disease-api --tail=50
```

```
kubectl delete -f deployment/
```

1.13 Run Prometheus and Grafana Monitoring Stack

The monitoring stack is started using Docker Compose.

```
docker compose -f docker-compose.monitoring.yml up --build
```

This starts:

- FastAPI model serving API
- Prometheus
- Grafana

Open the services:

```
FastAPI Swagger UI: http://127.0.0.1:8000/docs
FastAPI metrics:    http://127.0.0.1:8000/metrics
Prometheus:         http://127.0.0.1:9090
Grafana:            http://127.0.0.1:3000
```

Grafana login:

```
Username: admin
Password: admin
```

In Grafana, configure Prometheus as the data source using:

```
http://prometheus:9090
```

Prometheus scrapes the FastAPI `/metrics` endpoint and Grafana is used to visualize API request count, request duration, and total API traffic.

Example Prometheus/Grafana queries:

```
http_requests_total
```

```
sum(increase(http_request_duration_seconds_count[10m]))
```

```
rate(http_request_duration_seconds_sum[5m]) /
rate(http_request_duration_seconds_count[5m])
```

Generate sample API traffic:

```
$body = @{
    age = 63
    sex = 1
    cp = 3
    trestbps = 145
    chol = 233
    fbs = 1
    restecg = 0
    thalach = 150
    exang = 0
    oldpeak = 2.3
    slope = 0
    ca = 0
    thal = 1
} | ConvertTo-Json

1..10 | ForEach-Object { Invoke-RestMethod -Uri "http://127.0.0.1:8000/predict" -
Method Post -Body $body -ContentType "application/json" }
```

View API logs:

```
docker logs heart-disease-api-container
```

Stop the monitoring stack:

```
docker compose -f docker-compose.monitoring.yml down
```

1.14 Run GitHub Actions CI/CD Pipeline

The CI/CD workflow is defined in:

```
.github/workflows/ci.yml
```

The workflow runs automatically on push and pull request. It can also be manually triggered from:

```
GitHub Repository > Actions > MLOps CI Pipeline > Run workflow
```

The workflow performs:

- Dependency installation

- Linting with flake8
- Dataset download
- Model training
- Prediction pipeline validation
- Unit testing with pytest
- Docker image build
- Upload of model and report artifacts

Successful GitHub Actions screenshots are included in the report as CI/CD evidence.